

UNITY 3D CLIENT PORTFOLIO

팀 프로젝트 기술개발서

단순 발견 on/off 가 아닌 , 적의 감각을 이용한 점수기반 시스템 설계

대표 프로젝트

StealthActionGame — Enemy AI

1팀 김종찬 (적 AI 담당)



목차

01

p.03-04

개요와 설계 목표

프로젝트 소개 · 네 가지 설계 목표

02

p.05-06

아키텍처와 패턴 선택

State × Command, 인터페이스
경계

03

p.07-09

감지 시스템

의심 점수 · 근접 감지 · 청각

04

p.10-11

상태 설계

5 가지 상태와 전이 규칙

05

p.12-16

게임플레이 확장

유인 · 추적 · 공격 · 시체 ·
자유시간

06

p.17-20

트러블슈팅

암살 판정 · Idle 모션 · 감시
방향

07

p.21

데이터 주도 설계

EnemyAIData · 45 개 튜닝 값

08

p.22

핵심 요약

구조 · 게임플레이 · 문제 해결
원칙

StealthActionGame



제작 기간

2~3 주 [26.09.08 ~ 09.28]

참여 인원

4 명 (적 AI 담당)

언어 / 엔진

C# · Unity

기술 스택

NavMesh · DOTween

게임 설명

감옥을 배경으로 , 교도관의 시야와 소리를 피해 **잠입** · **암살** · **유인**으로 미션을 수행하는 3 인칭 스텔스 액션 게임입니다 .

구현 동기

스텔스 게임의 재미는 ' 적을 읽고 속이는 것 ' 에 있다고 보고 , **반응을 예측하고 역이용할 수 있는 적 AI** 를 목표로 감지 · 판단 · 행동 · 전투 전반을 맡았습니다 .



순찰 중인 교도관과 ? 인디케이터

설계 목표 → 설계 원칙

모든 설계 결정은 아래 네 가지 목표에서 출발했습니다

목표 (Why)

설계 원칙 (How)

1 플레이어가 적의 반응을 예측할 수 있어야 한다



즉시 발견 대신 단계적 의심 (점수 누적 + ? / ! 표시)

2 스텔스 전략 (유인 · 암살 · 우회) 이 확실히 통해야 한다



소리 유인은 확실하게, 등 뒤는 사각, 조사 지점 기준으로 발견

3 적 행동을 쉽게 늘리고 고칠 수 있어야 한다



'언제 바꿀지 (State)' 와 '무엇을 할지 (Command)' 를 분리

4 코드 수정 없이 밸런스를 맞출 수 있어야 한다



모든 수치를 ScriptableObject(EnemyAIData) 로 분리

전체 구조

적은 '플레이어'를 직접 모른다 — IDetectable / IDamageable 인터페이스로만 연결

입력 (외부 → 적)

PlayerDetectable

IDetectable 구현 · 위치 / 자세 / 소리

소리원 (병 · 경보기)

RegisterSound() 로 소리 전달

PrisonScheduleManager

자유시간 변경 이벤트

적 코어

EnemyAI

진입점 · 사운드 · 체크포인트 부활

EnemyStateMachine

상태 전환 · 공유 컨텍스트

EnemyPerception

시야 · 청각 · 시체 감지

EnemyMovement

NavMesh · Animator 래핑

EnemyAIData

ScriptableObject · 45 개 값

판단 · 실행

States (5)

Normal · Weak · Strong ·
Detected · Alert

Commands (4)

LookAround · Investigate ·
Chase · Attack

State 가 '언제' 를 판단하고
Command 가 '무엇을' 실행

출력 (적 → 외부)

IDamageable

플레이어 피해 전달

EnemyIndicator

? / ! 머리 위 표시

상태별 사운드

의심 · 발견 음성 · 효과음

암살 연동

EnemyObject · Kill UI 판정

왜 상태 패턴과 커맨드 패턴을 함께 썼나

문제

상태마다 '이동 · 두리번 · 확인' 코드가 반복되고, 상태 클래스 하나가 점점 비대해짐

원인

'언제 바꿀지 (전이 판단)'와 '무엇을 할지 (행동 실행)'가 한 클래스에 섞여 있음

선택

State 는 전이 판단만, Command 는 행동만 담당. 행동은 **Start · Tick · Cancel** 수명주기로 통일

결과

LookAround 하나를 **4 곳에서 재사용**. 상태를 빠져나갈 때 Cancel() 한 줄로 정리

두 인터페이스가 역할을 나눈다

```
interface IEnemyState
{ Enter(fsm); Tick(fsm); Exit(fsm); }

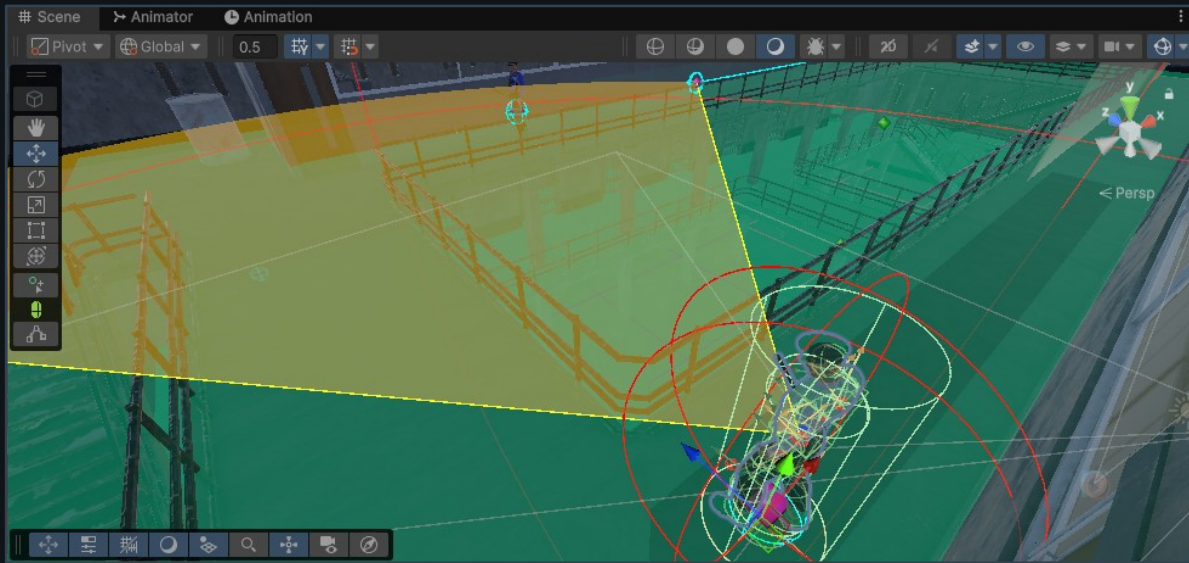
interface ICommand
{ Start(fsm, onComplete); Tick(); Cancel(); }
```

LookAroundCommand 재사용처

LookAround
Command

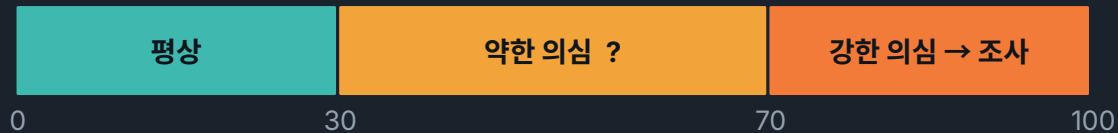
- Normal — 순찰 중 유히 두리번
- Weak — 의심 방향 주시
- Investigate — 도착 후 확인
- Alert — 시체 방향 경계

의심 점수 기반 감지 시스템



Scene 뷰 기즈모 — 시야 부채꼴 · 청각 반경 · 즉시감지 반경

점수 구간과 반응



WHY

보이는 순간 발견되면 스텔스 게임이 불공정하고 긴장감이 사라짐 . 보임 / 안보임 이진 판정으로는 거리 · 자세 · 은신 차이를 표현할 수 없음

HOW

0~100 의심 점수를 누적 . 거리 구간별 속도 (근 30 · 중 10 · 원 3 / 초) 에 **앞기 (×0.5)** · **은신 가중치를 곱함** . 안 보이면 감쇠 , 앞아서 숨으면 감쇠 ×4

RESULT

멀리서 잠깐 보이면 '?', 가까이 오래 보이면 조사 · 발견 → 플레이어가 **적의 반응을 읽고 대응**

예외 규칙 — 근접 즉시감지와 시야 판정

근접 즉시감지 (1.5m · 270° · 완전 노출)

문제

바로 앞에 있는데 점수가 쌓이길 기다리는 적은 어색함

선택

가까운 거리 + 전방 270° + 완전 노출이면 즉시 만점

이유

등 뒤 90°를 사각으로 비워 뒤에 다가오는 암살이 성립

시야 차단 판정 (Line of Sight)

문제

Raycast가 타겟 자신의 콜라이더에 맞아 벽이 없어도 '가려짐'으로 판정

선택

RaycastAll 후, 타겟 0.5m 이내에서 맞은 것은 무시

결과

콜라이더 구성과 상관없이 안정적인 시야 판정

시야 각도 요약

90°

일반 시야

거리 구간별로 점수 누적

270°

근접 즉시감지

1.5m 안 · 완전 노출일 때만

90°

후방 사각

암살 접근 경로로 남겨 둬

소리는 하나의 API 로 — RegisterSound()

문제

발소리 · 병 던지기 · 경보기가
각자 다른 방식이면 소리원이 늘
때마다 적 코드를 고쳐야 함

원인

소리를 '내는 쪽' 과 '해석하는
쪽' 이 묶여 있음

선택

**RegisterSound(위치, 강도,
돌발, 반경, 환경음)** 하나로
통일. 거리 감쇠 · 기억 ·
점수화는 Perception 내부에서

결과

새 소리원은 호출 한 줄. 경보기도
적 코드 수정 없이 추가

플레이어 발소리

던진 병 착지

경보기 (환경음)

RegisterSound()

소리 기억 잠금

약한 소리가 강한 소리의 위치를 덮어서 적이 엉뚱한 곳으로 가는 문제 → 잠금
시간 동안은 더 센 소리만 위치를 갱신

돌발 소리 즉시 만점

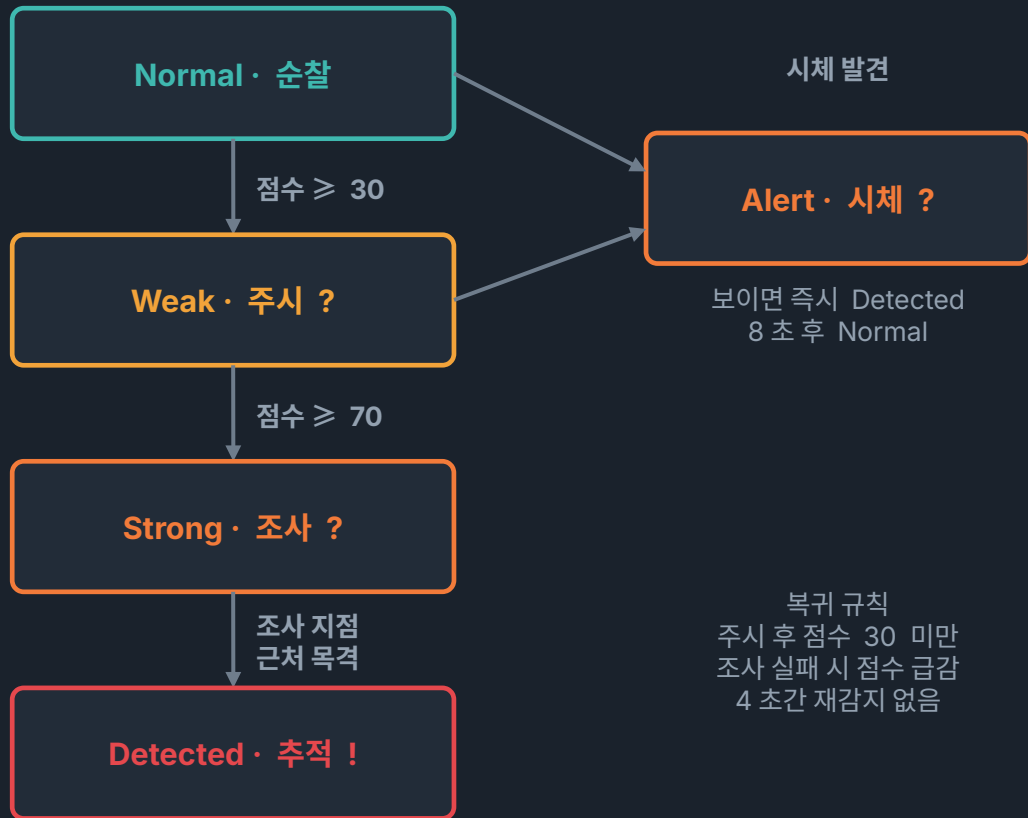
병 던지기 유인이 가끔 실패하면 플레이어가 전략을 믿지 못함 → 강한 의심
이상 세기면 즉시 만점으로 확실히 유인

상태별 설계 의도

각 상태는 플레이어에게 보여줄 ' 신호 ' 를 먼저 정하고 , 그 신호에 맞춰 동작을 설계했습니다

상태	플레이어가 보는 신호	핵심 동작	이렇게 만든 이유
Normal	표시 없음	순찰 , 확률적 두리번 / 고정 경비는 처음 방향 유지	가만히 선 적은 살아 있어 보이지 않음 → 확률적 유희로 패턴 암기를 어렵게
Weak	주황 ? (정지)	멈추고 의심 방향 주시 , 주시 중엔 강등 보류	점수가 바로 떨어져도 ' 들킬 뻔했다 ' 는 피드백을 최소 한 번은 보여줘야 함
Strong	빨강 ? (펄스)	소리 / 시야 중 더 강한 쪽 위치로 조사 이동	적이 어디를 의심하는지 움직임으로 보여줘야 플레이어가 역이용 (유인) 가능
Detected	빨강 ! (펄스)	실시간 추적 + 사거리 내 공격 , 암살 불가	정면으로 들킨 뒤 암살로 쉽게 빠져나가지 못하게 긴장 유지
Alert	빨강 ? (펄스)	시체 방향 경계 , 보이면 즉시 발견	암살에 ' 흔적 ' 이라는 대가를 붙여 시체 처리 플레이를 유도

5 단계 상태 전이



WHY

적이 어떤 상태인지 플레이어가 행동만 보고 알 수 있어야 함

HOW

Normal · Weak · Strong · Detected · Alert 로 분리 . 오를 땐 한 단계씩 , 내려올 땐 각 상태가 스스로 판단 . **Weak 는 주시가 끝날 때까지 강등 보류**

RESULT

'들킬 뻔했다' 는 피드백이 최소 한 번은 보이는 , 긴장 단계가 움직임으로 드러나는 AI

조사 중 발견 조건 — 유인 전략이 성립하도록

문제

조사하러 가는 도중, 먼 거리에서
플레이어가 보이거나 해도 바로
발견 처리됨

원인

'지금 보인다'만으로 발견을 판정
→ 소리로 유인해 두고 다른 길로
가는 전략이 무력화

선택

조사 지점 반경 안에서 보일 때만
발견, 그 외에는 조사를 계속.
반경 값은
investigateCatchRadius 로
조정

결과

경보기 · 병 던지기로 적을 끌어낸
뒤 **우회하는 플레이**가 성립



병 던지기 → 착지 소리로 유인



소리가 난 지점으로 향하는 교도관

관련 규칙

- 조사 위치 = 시야와 소리 중 점수가 더 높은 쪽
- 소리로 유인된 조사는 더 오래 머물 (기본 2.5 초 → 7 초)
- 조사 실패 시 점수를 크게 깎아 같은 곳을 반복 조사하지 않음

추적 불가 판정 — '보인다' 와 '갈 수 있다' 는 다르다

문제

난간 위 · 다른 층 등 갈 수 없는
곳에 플레이어가 있으면 적이
멈추거나 끝없이 쫓아옴

원인

'감지할 수 있다' 와 '도달할 수
있다' 를 같은 것으로 취급

선택

IsReachable() 로 경로가
끝까지 이어지는지 따로 검사.
일정 시간 막히면 '추적 불가'

결과

보이는 동안엔 경계 유지, 안
보이면 자연스럽게 포기 → 막힌
지형에서의 이상 행동 제거



철조망 너머 플레이어를 발견한 교도관

경로 있음

추적 + 사거리 안이면 공격

경로 없음 + 보임

포기 타이머 리셋 (인지 유지), 공격은 건너뛸

경로 없음 + 안 보임

즉시 놓친 것으로 처리 → Weak / Normal

공격은 3 단계로 — 피할 수 있는 공격

Windup 0.4s

대상 쪽으로 회전 · 공격 애니메이션 재생 · 이동 애니메이션 고정



Hit

휘두르기 사운드 · TakeDamage() 호출



Recovery 0.6s

자동 회전 복구 후 추적으로 복귀

왜 Windup 이 필요한가

공격 전 예고 동작이 있어야 플레이어가 피하거나 거리를 벌릴 여지가 생김

왜 회전을 직접 제어했나

NavMesh 가 회전을 덮어서 엉뚱한 방향으로 공격 → 자동 회전을 끄고 직접 보관

왜 이동 애니메이션을 고정했나

공격 중에도 NavMesh 의 잔여 속도가 남아 걷기 모션이 섞임 → Speed 를 0 으로 고정

왜 IDamageable 인가

적은 플레이어 체력 구현을 모름 → 더미로 먼저 테스트, 실제 체력 시스템은 교체만

시체 발견 — 암살에 대가를 붙이다

문제

암살에 아무 흔적이 남지 않으면
암살이 항상 정답이 되어
긴장감이 사라짐

원인

암살의 '결과'를 다른 적이
알아챌 방법이 없음

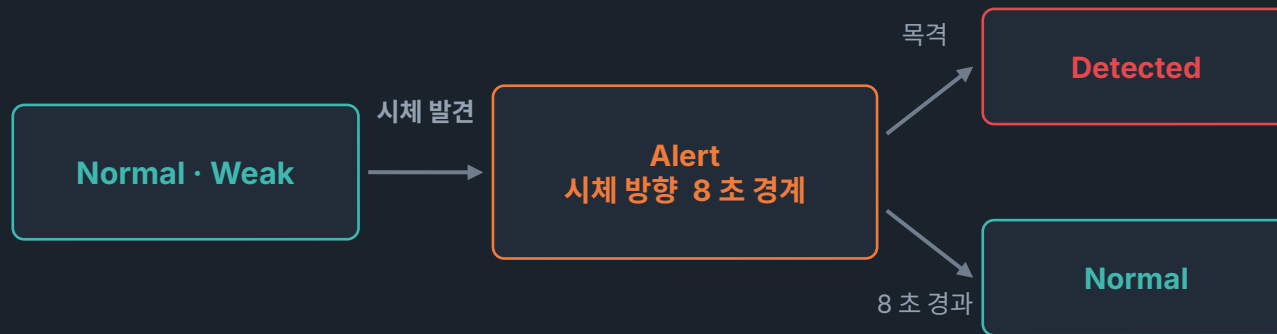
선택

시체 레이어를 감지하는 **Alert**
상태 추가. 같은 시체에는 **한**
번만 반응 (HashSet 기록)

결과

시체를 옮기고 숨기는 플레이에
의미가 생김. 같은 시체로
경계가 무한 반복되는 문제 방지

Alert 전이 흐름



Alert 상태 규칙

- Normal · Weak 상태에서만 시체를 감지
- 시체 방향을 8 초 동안 좌우로 경계
- 경계 중 플레이어가 보이면 즉시 Detected
- 벽 너머 시체는 시야 확인 후에만 반응

자유시간 구역 — 같은 AI 로 ' 허용 ' 과 ' 금지 ' 를 표현

문제

자유시간에 죄수 (플레이어) 가
돌아다니는 건 정상인데 적이 바로
의심 · 발견

원인

감지 로직이 게임 속 시간표 (규
칙) 를 모름

선택

PrisonScheduleManager
이벤트 구독 . 자유시간엔 감지
정지 , 의심 행동은 경고로
흡수하다 한도를 넘으면 발견

결과

같은 적 AI 로 **허용 구역**과 **금지
구역** 플레이를 모두 표현



자유시간 중 죄수에게 다가오는 교도관

자유시간 ?

→ 의심 행동

→ 경고음 + 점수 급감

→ 누적 ≥ 2 → 발견

예외 1 · 항상 경계하는 적

방을 지키는 교도관처럼 자유시간에도 감지를
멈추면 안 되는 적은 isAlwaysAlert 로 예외

예외 2 · 경보음은 도발이 아니다

경보기는 개인 행동이 아닌 환경을 →
자유시간이어도 경고로 흡수하지 않고 항상 조사

암살 판정 통합 — 보이는 것과 되는 것을 일치시키기

문제

'Kill' UI 가 떴는데 암살이 안 되거나, 반대로 UI 없이 암살되는 불일치

원인

UI 표시 조건과 암살 실행 조건을 각각 따로 계산

선택

CanTarget() 하나를 단일 진입점으로. UI 는 이벤트 구독으로만 표시 / 숨김

결과

보이는 것과 되는 것이 항상 일치. 발견 상태에선 암살 불가로 정면 전투 긴장 유지



판정 통과 시에만 뜨는 암살 UI

IsAssassinable
Detected 상태에서 false

CanTarget(victim)
거리 + 각도 + 암살 가능

실제 암살 실행
FindVictim()

Kill UI 표시
IsInteractable

Idle 모션 랜덤화 — 부드럽게보다 '언제'

문제 A · 모션이 중간에 끊김

증상

5.3 초짜리 대기 모션 도중에 두리번이 끼어들어 모션이 뚝 끊김

원인

특수 행동 시작 시점이 애니메이션 재생 상태와 무관

해결

normalizedTime 으로 루프 시작 / 끝 5% 구간에 왔을 때만 시작

문제 B · 모션이 멈춤 (0.9999999)

증상

부드럽게 바꾸려고 감쇠 부여 > IdleIndex 가 1에 달지 못하고 정지

원인

목표값에 접근할 뿐 도달하지 않음 . 블렌드 트리는 정수 임계값

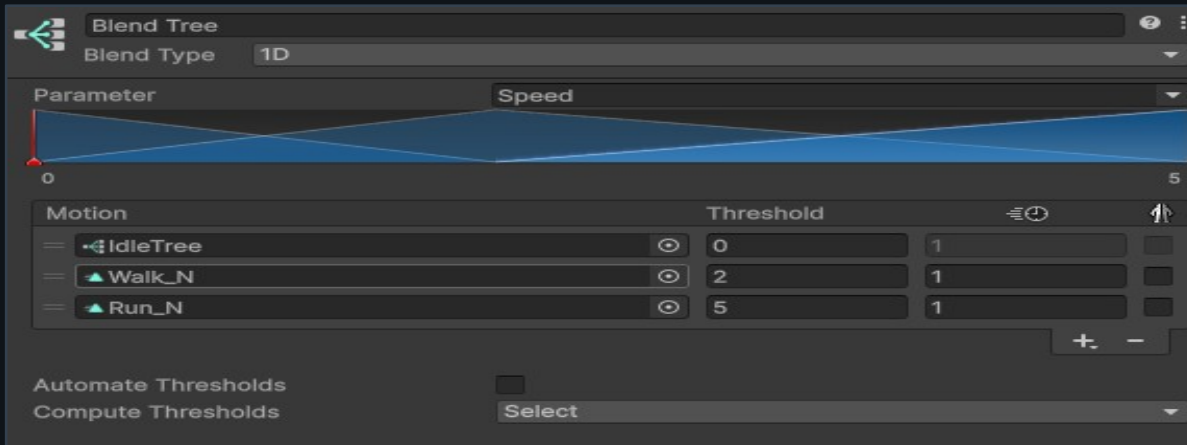
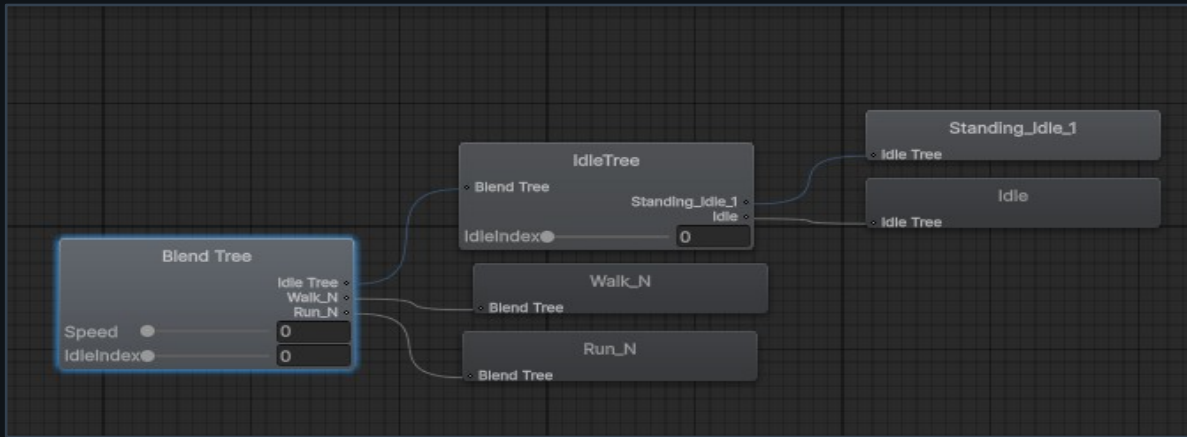
해결

감쇠 제거 . 전환 '시점' 을 루프 경계로 제어해 부드러움 확보

배운 점

값을 부드럽게 바꾸는 것보다 , 언제 바꿀지를 제어하는 것이 더 안정적이다 .

Idle 블렌드 트리 구조 — 해결 후의 모습



Animator — Speed 0 · 2 · 5 이동 트리 안에 IdleTree 중첩

구조

이동 1D 블렌드 트리 (Speed 0 · 2 · 5 = Idle · Walk · Run) 의 Speed 0 슬롯에 **IdleTree** 를 중첩하고, IdleIndex 파라미터로 대기 클립을 선택

전환 규칙

IdleIndex 는 **감식 없이 정수로 즉시 변경** . 두리번 (LookAround) 은 Idle 루프 경계에서만 시작

추가 처리

두리번 (회전) 중에는 IdleIndex 를 **0 으로 고정** 하고 , 끝나면 다시 랜덤 → 회전 모션과 다른 대기 모션이 섞이던 문제 해결

고정 경비의 감시 방향이 틀어지는 버그

문제

고정 경비가 조사 · 발견 후
자리로 돌아오면 처음 감시
방향이 아닌 엉뚱한 방향으로
고정

원인

복귀 이동 중에도 대기 타이머가
돌아 두리번이 시작 → 이동 중
회전값을 '기준 방향'으로 삼음

선택

HasArrived() 전에는 타이머
정지. 도착 후엔 스폰 시 저장한
SpawnRotation으로 방향 고정

결과

어떤 상태를 거쳐 돌아와도 원래
감시 방향 복원. 레벨
디자이너가 배치한 방향 = 감시
방향

Before

복귀 이동 중

타이머 만료 →
두리번 시작

이동 방향이
기준이 됨 ✕

After

복귀 이동 중
(타이머 정지)

HasArrived()
도착

SpawnRotation
복원 ✓

NormalState.TickSinglePointIdle()

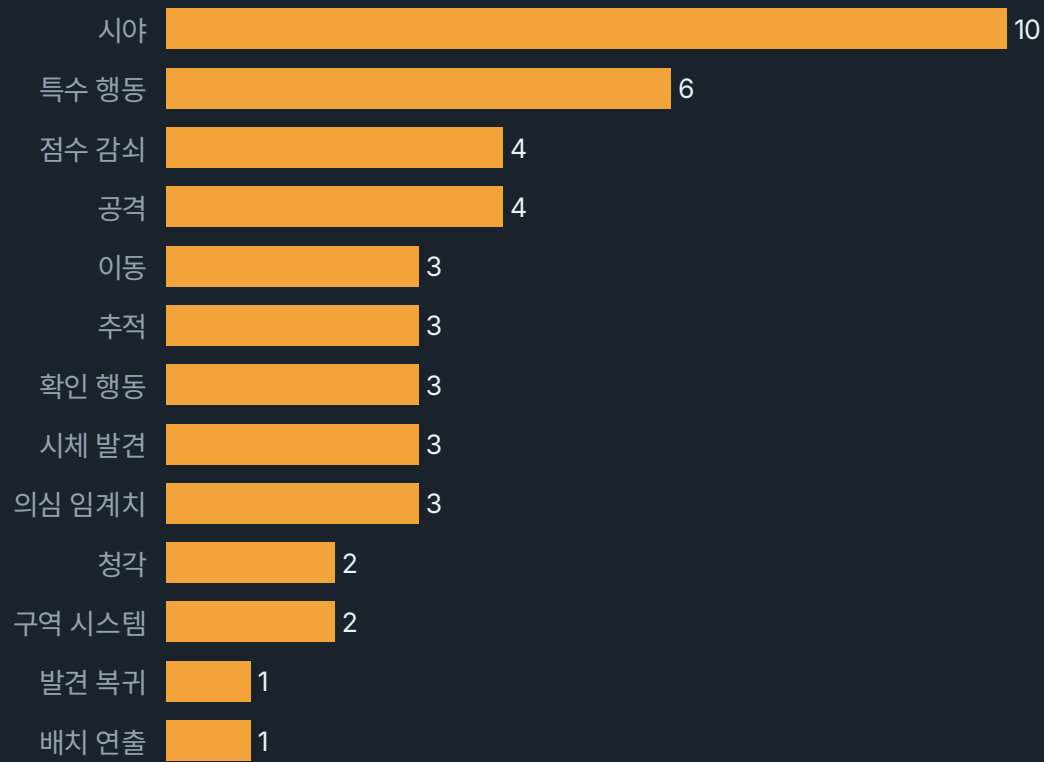
```
if (!fsm.Movement.HasArrived())
    return; // 복귀 중엔 타이머 정지
```

```
transform.rotation = Owner.SpawnRotation;
_singlePointTimer += Time.deltaTime;
```

데이터 주도 설계 — EnemyAIData

코드를 열지 않고 인스펙터에서 45 개 값을 조정

그룹별 파라미터 수



튜닝 속도

플레이 중 인스펙터 값이 바로 반영 → 코드 수정 · 재컴파일 없이 밸런스 조정

적 유형 분리

에셋만 바꿔 끼우면 다른 성격의 적 (예 : 시야 넓은 교도관)

팀 협업

레벨 디자이너가 직접 수치를 만질 수 있어 요청 - 수정 왕복 감소

값 순서 규칙

즉시감지 < 근거리 < 중거리 < 원거리
약한 의심 < 강한 의심 < 최대 점수

핵심 요약

01 '언제 (State)' 와 '무엇을 (Command)' 을 분리해 기능 추가에 강한 구조

02 점수 · 소리 · 조사 지점 규칙으로 읽을 수 있고, 역이용할 수 있는 적

03 트러블슈팅에서 얻은 원칙 — 값을 부드럽게 바꾸기보다 시점을 제어하라



THANK YOU

감사합니다

김종찬 · 적 AI 담당



https://github.com/Chan1605/StealthActionGame_N

